

TRANSFORMERS IN ACTION

Directions Futures
et Applications
Multimodales



BY
ES-SAOUDY Zakaria.

Contents

1	Introduction et cadre du travail	1
2	Architectures étudiées	1
2.1	Vision Transformer — ViT	1
2.2	iGPT — Image GPT	1
2.3	TAPAS — Table Question Answering	2
2.4	wav2vec 2.0 — Reconnaissance de la parole	2
2.5	CLIP — Contrastive Language–Image Pre-training	2
3	Mise à l’échelle des Transformers	2
3.1	Chronologie des grands modèles	2
3.2	Lois d’échelle	3
3.3	Défis et réponses techniques	3
4	Expériences : implémentation et résultats	4
4.1	Classification d’images — ViT via pipeline	4
4.2	Question Answering sur tables — TAPAS	4
4.3	Reconnaissance automatique de la parole — wav2vec 2.0	5
4.4	Alignement Vision-Texte — CLIP	5
5	Perspectives et directions ouvertes	6
6	Synthèse et conclusion	7
	Annexe — Code intégral du notebook	8

1 Introduction et cadre du travail

Ce travail pratique s'inscrit dans la continuité du cours de Deep Learning et porte sur l'étude approfondie des architectures Transformer. Le notebook `11_future-directions.ipynb`, chapitre conclusif de l'ouvrage *NLP with Transformers*, adopte une perspective prospective : il ne s'agit plus seulement d'appliquer des modèles, mais d'en comprendre les trajectoires d'évolution et les frontières actuelles.

Trois grandes questions structurent ce notebook :

1. **Jusqu'où peut-on scaler ?** La croissance exponentielle du nombre de paramètres obéit-elle à des lois prévisibles, et quels en sont les coûts ?
2. **Au-delà du texte.** Comment les Transformers s'adaptent-ils à la vision, aux structures tabulaires et à la parole ?
3. **Vers la multimodalité.** Que se passe-t-il lorsque l'on aligne plusieurs modalités dans un espace de représentation commun ?

L'implémentation repose intégralement sur Hugging Face Transformers, dont les pipelines de haut niveau permettent d'interroger des modèles pré-entraînés en quelques lignes.

2 Architectures étudiées

2.1 Vision Transformer — ViT

ViT (*Dosovitskiy et al., 2020*) est la transposition directe de l'encodeur BERT au domaine des images. Une image $H \times W$ est segmentée en N patches de taille $P \times P$, projetés linéairement en tokens. Le traitement est ensuite identique à celui du texte.

La séquence d'entrée est construite comme suit :

$$z_0 = [x_{\text{cls}}; \underbrace{x_p^1 E, x_p^2 E, \dots, x_p^N E}_{\text{patch embeddings}}] + E_{\text{pos}}, \quad E \in \mathbb{R}^{(P^2 C) \times D} \quad (1)$$

Un token de classification x_{cls} est préfixé à la séquence. En sortie de l'encodeur, sa représentation est transmise à une tête de classification linéaire. ViT atteint les performances état-de-l'art sur ImageNet uniquement lorsqu'il est pré-entraîné sur de très grandes collections (JFT-300M, ImageNet-21k) : c'est le coût de l'absence d'inductive bias spatial propre aux CNN.

2.2 iGPT — Image GPT

iGPT (*Chen et al., 2020*) étend GPT aux images en traitant les pixels comme des tokens. Une image basse résolution est quantifiée sur une palette réduite, et le modèle apprend à prédire le pixel suivant de façon autorégressive.

Ce travail illustre la *généralité du préentraînement autorégressif* : sans aucune supervision, iGPT extrait des représentations exploitables pour la classification et la complétion d'images, validant l'hypothèse que la modélisation de distribution est une forme puissante

d'apprentissage de représentation.

2.3 TAPAS — Table Question Answering

TAPAS (Herzig et al., 2020) enrichit BERT d'embeddings supplémentaires encodant les coordonnées 2D de chaque cellule (numéro de colonne, de ligne, rang dans le tableau). Le modèle apprend à sélectionner des cellules et à y appliquer un agrégateur numérique.

La réponse à une question peut être de deux natures : sélection directe (NONE) ou agrégation calculée (SUM, COUNT, AVERAGE). TAPAS est affiné sur WTQ (WikiTableQuestions) et SQA (Sequential Question Answering).

2.4 wav2vec 2.0 — Reconnaissance de la parole

wav2vec 2.0 (Baevski et al., 2020) encode un signal audio brut via une pile de convolutions, masque des segments de la représentation latente, puis entraîne un Transformer à reconstruire les unités masquées par apprentissage contrastif.

Ce préentraînement entièrement non supervisé réduit le besoin en transcriptions annotées à moins de 10 minutes. Le modèle facebook/wav2vec2-base-960h, affiné sur 960 h de LibriSpeech, est utilisé dans le notebook.

2.5 CLIP — Contrastive Language-Image Pre-training

CLIP (Radford et al., 2021) aligne un encodeur visuel (ViT ou ResNet) et un encodeur textuel (Transformer) dans un espace partagé, par apprentissage contrastif sur 400 millions de paires (image, texte) collectées sur le web.

La fonction de perte symétrique est :

$$\mathcal{L}_{\text{CLIP}} = -\frac{1}{2N} \sum_{i=1}^N \left[\log \frac{e^{\langle v_i, t_i \rangle / \tau}}{\sum_j e^{\langle v_i, t_j \rangle / \tau}} + \log \frac{e^{\langle t_i, v_i \rangle / \tau}}{\sum_j e^{\langle t_i, v_j \rangle / \tau}} \right] \quad (2)$$

Cette formulation fait de CLIP un classifieur *zero-shot* : les catégories sont décrites en langage naturel, sans fine-tuning sur le jeu cible.

3 Mise à l'échelle des Transformers

3.1 Chronologie des grands modèles

Le notebook encode dans `model_data` les jalons majeurs entre 2017 et 2021, visualisés en échelle logarithmique via `matplotlib`. Le tableau 1 synthétise ces données.

Table 1. Paramètres des grands modèles de langage (2017–2021)

Modèle	Publication	Paramètres	Facteur cumulé
Transformer	Juin 2017	213 M	$\times 1$
GPT	Juin 2018	110 M	—
BERT	Oct. 2018	340 M	$\times 1.6$
GPT-2	Fév. 2019	1.5 G	$\times 7$
T5	Oct. 2019	11 G	$\times 52$
Megatron	Sep. 2019	8.3 G	—
Turing-NLG	Fév. 2020	17 G	$\times 80$
GPT-3	Mai 2020	175 G	$\times 820$
GShard	Juin 2020	600 G	—
Switch-C	Jan. 2021	1.6 T	$\times 7\,500$

En moins de quatre ans, la taille des modèles a été multipliée par un facteur supérieur à 7 000 — une croissance que les axes linéaires ne peuvent représenter, d’où l’usage systématique de l’échelle log dans le notebook.

3.2 Lois d’échelle

Kaplan et al. (2020) ont établi empiriquement que la perte de test L suit une loi de puissance en chacun des axes : nombre de paramètres N , taille du corpus D , et budget de calcul C :

$$L(N) \approx \left(\frac{N_c}{N}\right)^{\alpha_N}, \quad L(C) \approx \left(\frac{C_c}{C}\right)^{\alpha_C} \quad (3)$$

Ces relations sont quasi-linéaires sur échelle log-log, avec des exposants $\alpha_N \approx 0.076$ et $\alpha_C \approx 0.050$. Conséquence pratique : pour un budget de calcul fixé, augmenter la taille du modèle est plus rentable qu’augmenter le nombre de passes d’entraînement. Henighan et al. (2020) ont validé ces lois sur d’autres modalités (images, code, mathématiques).

3.3 Défis et réponses techniques

La croissance des modèles pose trois défis structurels :

Complexité quadratique. L’auto-attention standard a un coût $\mathcal{O}(n^2)$ en mémoire et en temps. Deux familles de solutions émergent :

- *Sparse Attention* (Longformer, BigBird) : des masques de parcimonie (locale, stride, globale) réduisent à $\mathcal{O}(n)$ ou $\mathcal{O}(n \log n)$.
- *Linearized Attention* : une décomposition noyau réécrit le produit scalaire sous la forme $\phi(Q)(\phi(K)^\top V)$, ramenant la complexité à $\mathcal{O}(n \cdot d^2)$ indépendamment de la longueur.

Coût environnemental. L’entraînement de GPT-3 est estimé à plusieurs centaines de tonnes de CO₂ équivalent. L’efficacité computationnelle est devenue un critère de conception à part entière.

Biais et sécurité. Les modèles à grande échelle amplifient les biais statistiques des corpus web, posant des problèmes de fiabilité et d’équité que le scaling seul ne résout pas.

4 Expériences : implémentation et résultats

4.1 Classification d'images — ViT via pipeline

Le pipeline image-classification de Hugging Face instancie ViT sans configuration explicite. En l'absence de l'image cible, le code génère un fallback synthétique de 100×100 pixels.

```

1 from PIL import Image
2 from transformers import pipeline
3 import pandas as pd
4
5 try:
6     image = Image.open("images/doge.jpg")
7 except FileNotFoundError:
8     image = Image.new('RGB', (100, 100), color='red')
9
10 # Pipeline par défaut : google/vit-base-patch16-224
11 image_classifier = pipeline("image-classification")
12 preds = image_classifier(image)
13 print(pd.DataFrame(preds))

```

Listing 1. Classification d'images avec ViT (cellule 23–24)

Comportement observé. Le modèle `google/vit-base-patch16-224` est téléchargé automatiquement (avertissement `"defaulted to google/vit-base-patch16-224"`). Les prédictions sont retournées sous la forme `[[label, score]]` triées par confiance décroissante.

4.2 Question Answering sur tables — TAPAS

Un tableau de chapitres de livre est construit manuellement et soumis à quatre questions de natures distinctes (sélection, somme, moyenne, comptage).

```

1 import pandas as pd
2 from transformers import pipeline
3
4 book_data = [
5     {"chapter": 0, "name": "Introduction",
6      "start_page": 1, "end_page": 11},
7     {"chapter": 1, "name": "Text classification",
8      "start_page": 12, "end_page": 48},
9     {"chapter": 2, "name": "Named Entity Recognition",
10      "start_page": 49, "end_page": 73},
11     {"chapter": 3, "name": "Question Answering",
12      "start_page": 74, "end_page": 120},
13     {"chapter": 4, "name": "Summarization",
14      "start_page": 121, "end_page": 140},
15     {"chapter": 5, "name": "Conclusion",
16      "start_page": 141, "end_page": 144},
17 ]
18 table = pd.DataFrame(book_data)
19 table['number_of_pages'] = table['end_page'] - table['start_page']
20 table = table.astype(str)
21
22 table_qa = pipeline("table-question-answering")
23 queries = [
24     "What's the topic in chapter 4?",
25     "What is the total number of pages?",
26     "On which page does the chapter about question-answering start?",
27     "How many chapters have more than 20 pages?"
28 ]
29 preds = table_qa(table, queries)

```

```

30 for q, p in zip(queries, preds):
31     print(q, "=>", p["answer"])

```

Listing 2. Pipeline TAPAS sur tableau structuré (cellule 28–32)

Les résultats d'exécution sont récapitulés dans le tableau 2.

Table 2. Résultats TAPAS (*google/tapas-base-finetuned-wtq*)

Question	Réponse produite	Agrégateur
Thème du chapitre 4 ?	Summarization	NONE
Total des pages ?	SUM > 10, 36, 24, 46, 19, 3	SUM
Page de début du QA ?	AVERAGE > 74	AVERAGE
Chapitres de plus de 20 pages ?	COUNT > 1, 2, 3	COUNT

La réponse à la question de sélection directe (chapitre 4) est juste. Les agrégations numériques sont correctes ($10 + 36 + 24 + 46 + 19 + 3 = 138$ pages via SUM). La question de comptage retourne les indices des chapitres éligibles plutôt qu'une valeur scalaire — limite connue de la version *base* de TAPAS.

4.3 Reconnaissance automatique de la parole — wav2vec 2.0

```

1 from transformers import pipeline
2 from datasets import load_dataset
3 import soundfile as sf
4
5 # Modèle par défaut : facebook/wav2vec2-base-960h
6 asr = pipeline("automatic-speech-recognition")
7
8 # Chargement d'un échantillon SUPERB ASR
9 try:
10     ds = load_dataset("superb", "asr", split="validation[:1]")
11     def map_to_array(batch):
12         speech, _ = sf.read(batch["file"])
13         batch["speech"] = speech
14         return batch
15     ds = ds.map(map_to_array)
16     ds.set_format("numpy")
17     pred = asr(ds[0]["speech"])
18     print(pred)
19     # => {'text': 'MISTER QUILTER IS THE APOSTLE OF THE MIDDLE ...'}
20 except Exception as e:
21     print(f"Erreur chargement dataset : {e}")

```

Listing 3. Pipeline ASR et chargement audio (cellule 36–41)

Exécution locale. Le chargement du dataset superb échoue avec l'erreur Invalid HF URI 'hf://datasets/superb@...', imputable à une version de la bibliothèque datasets non compatible avec l'identifiant de dépôt. Ce dysfonctionnement est environnemental et non algorithmique : le pipeline ASR s'initialise correctement (facebook/wav2vec2-base-960h, 960 h de LibriSpeech). En environnement Colab ou Kaggle, le flux complet est opérationnel.

4.4 Alignement Vision-Texte — CLIP

```

1 from transformers import CLIPProcessor, CLIPModel

```

```

2 from PIL import Image
3 import torch
4
5 clip_ckpt = "openai/clip-vit-base-patch32"
6 model = CLIPModel.from_pretrained(clip_ckpt)
7 processor = CLIPProcessor.from_pretrained(clip_ckpt)
8
9 image = Image.open("images/optimusprime.jpg")
10 texts = ["a photo of a transformer",
11          "a photo of a robot",
12          "a photo of agi"]
13
14 inputs = processor(text=texts, images=image,
15                   return_tensors="pt", padding=True)
16 with torch.no_grad():
17     outputs = model(**inputs)
18
19 probs = outputs.logits_per_image.softmax(dim=1)
20 for t, p in zip(texts, probs[0]):
21     print(f"{t:35s} -> {p.item():.4f}")

```

Listing 4. *Inférence zero-shot avec CLIP (cellule 52–54)*

Exécution locale. Les 398 fichiers de poids sont téléchargés, mais l’instanciation du modèle produit une erreur de compatibilité de version. En environnement stable, `probs` est un tenseur 1×3 dont les valeurs reflètent la similarité cosinus entre l’image et chacune des descriptions textuelles normalisées.

5 Perspectives et directions ouvertes

Le notebook referme le cours en signalant quatre directions qui prolongent directement les travaux présentés.

DALL·E. Ramesh et al. (2021) entraînent un décodeur autorégressif de 12 milliards de paramètres sur des paires (image, texte) via un tokenizer visuel discret (dVAE). CLIP sert d’oracle pour reclasser les images générées. Ce modèle préfigure les systèmes de génération conditionnée (DALL·E 2, Stable Diffusion) qui dominent aujourd’hui la génération d’images.

LayoutLM. Xu et al. (2020) augmentent BERT d’embeddings de position 2D extraits des boîtes de texte d’un document numérisé. LayoutLMv2 ajoute une branche visuelle via ResNeXt. Cette architecture atteint l’état de l’art sur l’extraction d’information dans les formulaires, factures et documents financiers.

wav2vec-U. Baevski et al. (2021) franchissent la barrière de la supervision : wav2vec-U entraîne un système ASR sans aucune transcription, en alignant des segments acoustiques à des phonèmes via un discriminateur GAN. C’est une percée directe pour les langues à faibles ressources.

Mixture of Experts (MoE). Switch-C (Fedus et al., 2021) dépasse le trillion de paramètres grâce à la conditionnalité sparse : à chaque token, seul un sous-ensemble d’experts est activé. Le coût de calcul est ainsi décorrélié du nombre total de paramètres, rendant les très grands modèles économiquement accessibles.

6 Synthèse et conclusion

Ce travail pratique illustre une transition majeure dans l'histoire des architectures profondes : le Transformer, conçu pour la traduction automatique, est devenu un framework généraliste capable d'unifier texte, image, parole et données structurées dans un cadre computationnel commun.

Quatre enseignements ressortent de l'analyse du notebook :

- **La scalabilité est prévisible, pas gratuite.** Les lois de puissance de Kaplan donnent une carte de navigation pour l'allocation des ressources, mais la croissance illimitée des modèles se heurte à des contraintes énergétiques et économiques incontournables.
- **L'attention efficiente ouvre de nouveaux horizons.** La réduction de $\mathcal{O}(n^2)$ à $\mathcal{O}(n)$ débloque le traitement de séquences longues (documents, génomes, séquences audio), élargissant considérablement le champ d'application.
- **La multimodalité émerge de l'alignement contrastif.** CLIP démontre qu'un espace de représentation partagé, appris sans supervision explicite, suffit à réaliser une classification zero-shot compétitive et à servir de fondation à la génération d'images.
- **La robustesse de l'implémentation compte autant que l'algorithme.** Les erreurs rencontrées en environnement local (incompatibilités de version, URIs de dataset non résolus) n'invalident pas l'architecture sous-jacente ; elles soulignent l'importance de la reproductibilité et de la gestion des dépendances dans un code de production.

En définitive, ce chapitre 11 constitue une boussole pour les praticiens : les Transformers ne sont pas une solution figée mais un paradigme en constante expansion, dont GPT-4, Gemini et les architectures diffusion illustrent aujourd'hui la profondeur industrielle.

Annexe — Code intégral du notebook

A.1 Installation des dépendances

```

1 import subprocess, sys, warnings
2
3 packages = ["transformers", "torch", "pillow", "datasets", "soundfile"]
4 for package in packages:
5     try:
6         __import__(package)
7     except ImportError:
8         subprocess.check_call([sys.executable, "-m", "pip",
9                                "install", "-q", package])
10 warnings.filterwarnings('ignore')
```

A.2 Visualisation de la croissance des modèles

```

1 import pandas as pd
2 import matplotlib.pyplot as plt
3
4 model_data = [
5     {'date': '12-06-2017', 'name': 'Transformer', 'size': 213e6},
6     {'date': '11-06-2018', 'name': 'GPT', 'size': 110e6},
7     {'date': '11-10-2018', 'name': 'BERT', 'size': 340e6},
8     {'date': '14-02-2019', 'name': 'GPT-2', 'size': 1.5e9},
9     {'date': '23-10-2019', 'name': 'T5', 'size': 11e9},
10    {'date': '17-09-2019', 'name': 'Megatron', 'size': 8.3e9},
11    {'date': '13-02-2020', 'name': 'Turing-NLG', 'size': 17e9},
12    {'date': '30-06-2020', 'name': 'GShard', 'size': 600e9},
13    {'date': '28-05-2020', 'name': 'GPT-3', 'size': 175e9},
14    {'date': '11-01-2021', 'name': 'Switch-C', 'size': 1.571e13},
15 ]
16
17 df = pd.DataFrame.from_records(model_data)
18 df["date"] = pd.to_datetime(df["date"], dayfirst=True)
19
20 fig, ax = plt.subplots(1, 1, figsize=(12, 4))
21 df.plot(x="date", y="size", kind="scatter", s=15, ax=ax)
22 ax.set_yscale("log")
23 ax.set_xlabel("Release date")
24 ax.set_ylabel("Number of parameters")
25 ax.grid(True)
26 plt.tight_layout()
27 plt.show()
```

A.3 ViT — Classification d'images

```

1 from PIL import Image
2 from transformers import pipeline
3 import pandas as pd
4
5 try:
6     image = Image.open("images/doge.jpg")
7 except FileNotFoundError:
8     image = Image.new('RGB', (100, 100), color='red')
9
10 image_classifier = pipeline("image-classification")
11 preds = image_classifier(image)
12 print(pd.DataFrame(preds))
```

A.4 TAPAS — Question Answering sur tables

```

1 import pandas as pd
2 from transformers import pipeline
```

```

3
4 book_data = [
5     {"chapter": 0, "name": "Introduction",
6      "start_page": 1, "end_page": 11},
7     {"chapter": 1, "name": "Text classification",
8      "start_page": 12, "end_page": 48},
9     {"chapter": 2, "name": "Named Entity Recognition",
10      "start_page": 49, "end_page": 73},
11     {"chapter": 3, "name": "Question Answering",
12      "start_page": 74, "end_page": 120},
13     {"chapter": 4, "name": "Summarization",
14      "start_page": 121, "end_page": 140},
15     {"chapter": 5, "name": "Conclusion",
16      "start_page": 141, "end_page": 144},
17 ]
18 table = pd.DataFrame(book_data)
19 table['number_of_pages'] = table['end_page'] - table['start_page']
20 table = table.astype(str)
21
22 table_qa = pipeline("table-question-answering")
23 queries = [
24     "What's the topic in chapter 4?",
25     "What is the total number of pages?",
26     "On which page does the chapter about question-answering start?",
27     "How many chapters have more than 20 pages?"
28 ]
29 preds = table_qa(table, queries)
30 for query, pred in zip(queries, preds):
31     print(query, "=>", pred["answer"])

```

A.5 wav2vec 2.0 — Reconnaissance de la parole

```

1 from transformers import pipeline
2 from datasets import load_dataset
3 import soundfile as sf
4
5 asr = pipeline("automatic-speech-recognition")
6
7 ds = load_dataset("superb", "asr", split="validation[:1]")
8
9 def map_to_array(batch):
10     speech, _ = sf.read(batch["file"])
11     batch["speech"] = speech
12     return batch
13
14 ds = ds.map(map_to_array)
15 ds.set_format("numpy")
16 pred = asr(ds[0]["speech"])
17 print(pred)
18 # => {'text': 'MISTER QUILTER IS THE APOSTLE OF THE MIDDLE CLASSES ...'}

```

A.6 CLIP — Alignement Vision-Texte

```

1 from transformers import CLIPProcessor, CLIPModel
2 from PIL import Image
3 import torch
4
5 clip_ckpt = "openai/clip-vit-base-patch32"
6 model = CLIPModel.from_pretrained(clip_ckpt)
7 processor = CLIPProcessor.from_pretrained(clip_ckpt)
8
9 image = Image.open("images/optimusprime.jpg")
10 texts = ["a photo of a transformer",
11         "a photo of a robot",
12         "a photo of agi"]

```

```
13
14 inputs = processor(text=texts, images=image,
15                     return_tensors="pt", padding=True)
16 with torch.no_grad():
17     outputs = model(**inputs)
18
19 probs = outputs.logits_per_image.softmax(dim=1)
20 for text, prob in zip(texts, probs[0]):
21     print(f"{text:35s} -> {prob.item():.4f}")
```